

Normalización de bases de datos

De una tabla con problemas a varias tablas bien formadas

Semana 04 - Sesión 08

Rel Guzman



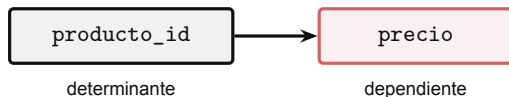
**Universidad
Tecnológica
del Perú**

Logro de la sesión

Al finalizar, podrás mirar una tabla **mal diseñada** y decir **qué regla incumple**; con eso sabrás **en qué forma normal está** y cómo corregirla.

Dependencia funcional

Al fijar el valor de una columna, queda determinado **un único** valor de otra.



Se escribe `producto_id → precio`

Se lee: “**sabiendo el producto, sé su precio**”.

La prueba de la sesión pasada: tapa dos columnas; si al fijar la primera siempre queda un único valor en la segunda, la primera manda sobre la segunda.

Tipo	En qué consiste	Ejemplo de la tienda
Completa (plena)	Se necesita el determinante entero	{pedido_id, producto_id} → cantidad
Transitiva	Se llega pasando por una tercera columna	pedido_id → cliente_id → cliente_nombre
Trivial	Lo pedido ya venía incluido en lo dado	{pedido_id, producto_id} → pedido_id

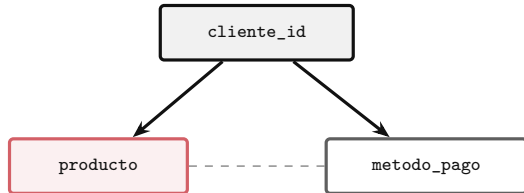
Hoy usamos las dos primeras para ponerle nombre y número a cada paso de separar una tabla.

Repaso: la multivaluada, en un ejemplo simple

Un cliente puede comprar **varios productos** y tener **varios métodos de pago**. Las dos listas son independientes entre sí:

cliente_producto_pago

cliente_id	producto	metodo_pago
40285571	Audífonos	Tarjeta
40285571	Audífonos	PayPal
40285571	Cargador	Tarjeta
40285571	Cargador	PayPal



independientes entre sí

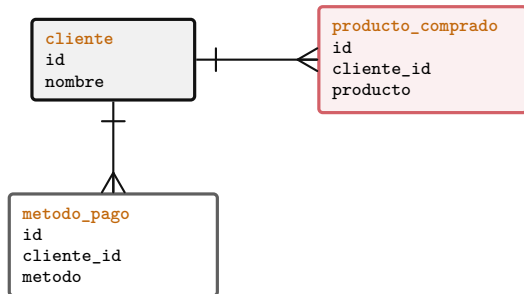
Las dos flechas salen del mismo dato: el **producto** comprado en un pedido depende del `cliente_id`, y el **método de pago** usado en ese pedido **también** depende del `cliente_id`, pero no uno de otro.

Con más clientes, la mezcla crece rápido

cliente_producto_pago

cliente_id	producto	metodo_pago
40285571	Audífonos	Tarjeta
40285571	Audífonos	PayPal
40285571	Cargador	Tarjeta
40285571	Cargador	PayPal
16458744	Mouse	Tarjeta
16458744	Teclado	Tarjeta

La salida de la dependencia multivaluada:
separar en dos tablas hermanas, cada una con
cliente_id, sin ninguna relación entre ellas.



Cada cliente_id aparece **muchas** veces en producto_comprado y **muchas** veces en metodo_pago: dos relaciones 1:N que nunca se cruzan entre sí.

Antes de separar: la tabla única de la tienda

Así estaba la tabla de la tienda la sesión pasada, con una fila por cada línea de venta:

pedido_id	cliente_id	cliente_nombre	producto_id	producto_nombre	precio	cantidad
1	40285571	Raúl	1	Audífonos	89.00	2
1	40285571	Raúl	2	Cargador	35.00	1
2	16458744	Mariana	1	Audífonos	89.00	1
2	16458744	Mariana	3	Mouse	45.00	3
3	36547781	Elena	2	Cargador	35.00	2

El problema que ya conocíamos

El producto 1 aparece dos veces, y con él su nombre y su precio **89.00** repetidos. De esta tabla salieron las cuatro que separamos en la sesión pasada.

Repaso 3: así quedó la tienda, separada

La sesión pasada separamos la tabla única en cuatro, siguiendo las dependencias:

cliente		pedido		producto		detalle_pedido		
id	nombre	id	cli_id	id	precio	pedido_id	producto_id	cantidad
40285571	Raúl	1	40285571	1	89.00	1	1	2

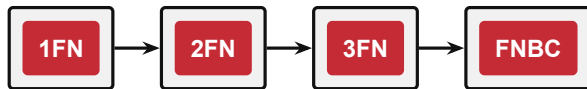
Lo que todavía no dijimos

Separamos por instinto, siguiendo las dependencias. Hoy vamos a ponerle **nombre y número** a cada paso: eso es la **normalización**.

Normalización

Normalización

Un proceso en pasos: revisar una tabla contra una lista de reglas, y si falla alguna, separarla. Cada paso es una **forma normal**.



Son **acumulativas**: para estar en 3FN una tabla también debe cumplir 1FN y 2FN. Cada forma corrige un problema que la anterior no alcanza a ver.

En palabras simples

Es como revisar un carro antes de un viaje: primero que tenga las cuatro llantas (1FN), luego que frene bien (2FN), luego que no se recaliente (3FN). Pasar la primera revisión no garantiza pasar la segunda.

Antes de normalizar: una tabla con tres problemas

La tienda, otra vez en una sola tabla, pero ahora con los teléfonos del cliente:

pedido_id	cliente_id	cliente_nombre	cliente_telefonos	producto_id	producto_nombre	precio
1	40285571	Raúl	987654321, 912345678	1	Audífonos	89.00
1	40285571	Raúl	987654321, 912345678	2	Cargador	35.00
2	16458744	Mariana	998877665	1	Audífonos	89.00

Guarda esta tabla en la cabeza: en las próximas diapositivas la vamos a corregir **un problema a la vez**, y cada corrección tiene nombre.

Primera forma normal (1FN)

Cada celda guarda **un solo valor**, y no hay columnas repetidas para la misma idea (como `telefono1`, `telefono2`).

Rompe la 1FN

`cliente_telefonos`: “**987654321**,
912345678” son dos teléfonos metidos en una sola celda.

Cumple la 1FN

Una tabla `telefono(id, numero, cliente_id)`, con una **fila** por cada número.

Cómo se detecta: si para leer una celda necesitas *separarla* tú mismo (por comas, por espacios) antes de usarla, esa celda no es atómica.

Ojo con esto

No es solo por comas. Guardar `telefono1` y `telefono2` como dos columnas también rompe la 1FN: sigue siendo la misma idea repetida, y un tercer teléfono no tiene dónde ir.

Completa: ¿está en 1FN?

Columna	¿1FN?
correo: "raul@mail.com"	_____
direcciones: "Av. Arequipa 100; Jr. Lima 250"	_____
categoria1, categoria2, categoria3	_____

Pista: pregúntate si, para usar el dato, alguien tendría que cortar el texto o mirar varias columnas que dicen lo mismo.

Con la 1FN resuelta, así queda la tabla

Cada teléfono ahora vive en su propia fila, en su propia tabla: tabla principal (sin cliente_telefonos)

pedido_id	cliente_id	cliente_nombre	producto_id	producto_nombre	precio
1	40285571	Raúl	1	Audífonos	89.00
1	40285571	Raúl	2	Cargador	35.00
2	16458744	Mariana	1	Audífonos	89.00

telefono

numero	cliente_id
987654321	40285571
912345678	40285571
998877665	16458744

Pero la tabla principal todavía tiene problemas

Fíjate en producto_nombre y precio: se repiten. Esos son los problemas de las siguientes dos diapositivas.

2FN Segunda forma normal: depender de la clave completa

Segunda forma normal (2FN)

Está en 1FN, y además **ninguna columna se conforma con la mitad de la clave**: si de verdad la necesita completa, tiene que usar las dos partes juntas.

En la tabla principal la clave es {pedido_id, producto_id}: dos columnas juntas.

En palabras simples

Es como un asiento de cine: “fila G” sola no ubica a nadie, y “asiento 12” solo tampoco. Hacen falta las **dos partes juntas**. Si para ubicar un dato te alcanza con **una sola** de las dos, esa columna no depende de la clave completa.

Ojo con esto

Esto **solo aplica cuando la clave tiene dos o más columnas**. Si la clave es una sola columna (como id), la tabla ya está automáticamente en 2FN.

Dos filas con el **mismo** producto_id pero **distinto** pedido_id:

pedido_id	producto_id	producto_nombre	precio
1	1	Audífonos	89.00
2	1	Audífonos	89.00

El pedido cambió y el precio no

Cambiamos pedido_id (de 1 a 2) y precio **se quedó igual**. Eso quiere decir que pedido_id no hacía falta: con producto_id solo ya alcanzaba.

Conclusión: producto_nombre y precio le bastan con **una sola parte** de la clave, no con las dos. Por la misma prueba, cliente_nombre le basta con pedido_id solo.

Paso 2: a esas columnas les sobra la mitad de la clave

producto_nombre y precio no necesitan el pedido para saberse, con producto_id basta:



Por qué molesta

Audífonos aparece en dos pedidos, y con él su precio 89.00. Si sube a 99.00 hay que corregir **cada pedido** donde aparece; olvidar uno deja dos precios distintos.

La corrección: sacar producto_nombre y precio a su propia tabla **producto**(id, nombre, precio), igual que hicimos la sesión pasada.

Con la 2FN resuelta, así queda la tabla

Ya en 2FN: cada tabla tiene una clave, y todo lo demás depende de ella completa:

pedido

id	cliente_id
1	40285571
2	16458744

producto

id	nombre	precio
1	Audífonos	89.00
2	Cargador	35.00

detalle_pedido

pedido_id	producto_id	cantidad
1	1	2
1	2	1
2	1	1

cliente

id	nombre
40285571	Raúl
16458744	Mariana

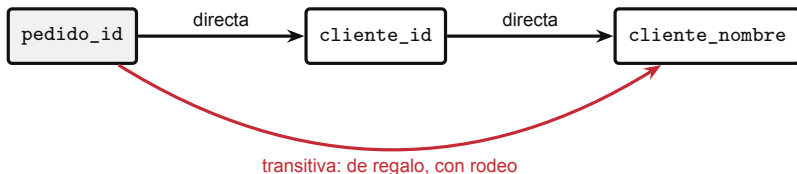
Todavía queda un problema, y no está en detalle_pedido

Mira las tablas **pedido** y **cliente**: para llegar al nombre de Raúl desde el pedido, hay que **pasar por** cliente_id. Eso ya lo vimos la sesión pasada con otro nombre.

Tercera forma normal (3FN)

Está en 2FN, y además **ninguna columna depende de otra que no sea la clave**. Nada de dependencias transitivas.

Si cliente_nombre viviera dentro de **pedido**, la cadena sería:



La regla práctica: si para llegar a un dato hay que pasar por una columna que **no es la clave**, ese dato pertenece a otra tabla. Por eso cliente_nombre vive en **cliente**, no en **pedido**.

En palabras simples

Es como pedir tu vuelto: el cajero no te lo da sabiendo tu nombre, pasa primero por la caja. **pedido** tampoco debería “saber” el nombre del cliente sin pasar por cliente_id.

Completa: encuentra la dependencia transitiva

La tabla **producto** de un catálogo, con el proveedor incluido:

id	nombre	proveedor_id	proveedor_nombre	proveedor_ciudad
1	Audífonos	10	SonidoTop	Lima
2	Cargador	10	SonidoTop	Lima
3	Mouse	11	ClickPerú	Arequipa

Completa la cadena, igual que en el ejemplo anterior ($\text{pedido_id} \rightarrow \text{cliente_id} \rightarrow \text{cliente_nombre}$): en el primer espacio va la columna que **id** determina **directamente** y que no es la clave; en el segundo, la columna a la que solo se llega **pasando por** esa primera.

id $\rightarrow$ _____ $\rightarrow$ proveedor_ciudad

¿Qué tabla nueva hace falta, y qué columna se queda en **producto** como clave foránea?

Con la 3FN resuelta, así queda el catálogo

producto

id	nombre	proveedor_id
1	Audífonos	10
2	Cargador	10
3	Mouse	11

proveedor

id	nombre	ciudad
10	SonidoTop	Lima
11	ClickPerú	Arequipa

Con esto, la tabla **producto** ya está en 3FN

Toda columna depende **directamente** de `id` y de nada más. Lima queda escrito una sola vez, junto a su proveedor.

En la práctica, **la gran mayoría** de las tablas bien diseñadas se quedan aquí: 3FN alcanza casi siempre.

En un instituto, cada profesor dicta un solo curso, pero un curso puede tener varios profesores (uno por sección):

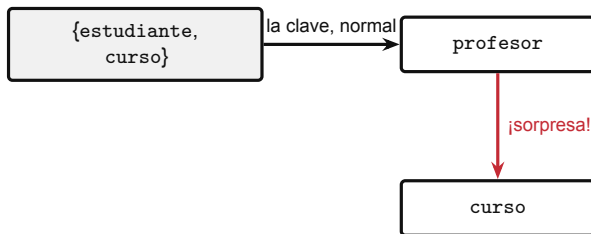
estudiante	curso	profesor
Ana	Base de Datos	García
Luis	Base de Datos	García
Ana	Redes	Pérez

La clave de esta tabla es {estudiante, curso}: ya está en 3FN. Pero prueba con los datos, tapando solo profesor y curso:

Algo curioso

Cada vez que profesor dice **García**, curso dice **Base de Datos**, las dos veces. profesor por sí solo **también** determina algo. ¿Casualidad, o hay una segunda dependencia escondida?

La tabla tiene, en realidad, **dos** dependencias funcionales:



Forma normal de Boyce-Codd (FNBC)

Está en 3FN, y además **todo determinante es una clave candidata**: una columna (o combinación) que también podría identificar la fila por sí sola.

profesor determina a curso, pero profesor solo **no** identifica la fila (no dice de qué estudiante se trata): no es clave candidata. Por eso esta tabla **no** cumple FNBC, aunque sí cumpla 3FN.

Sacamos la dependencia sobrante ($\text{profesor} \rightarrow \text{curso}$) a su propia tabla:

profesor	
nombre	curso
García	Base de Datos
Pérez	Redes

inscripcion	
estudiante	profesor
Ana	García
Luis	García
Ana	Pérez

Ahora `curso` vive solo en `profesor`, junto a su determinante real. En `inscripcion`, `profesor` ya no determina nada por su cuenta: **todo** determinante vuelve a ser clave candidata.

Ojo con esto

Este caso es **raro**: hace falta que una tabla tenga dos claves candidatas que se pisan. Si nunca lo ves, ya estás en FNBC en cuanto llegas a 3FN. Por eso casi no se usa en el trabajo diario.

Preguntas



Universidad
Tecnológica
del Perú



Forma	Exige	Se rompe cuando...
1FN	Celdas con un solo valor, sin columnas repetidas	una celda guarda varios teléfonos
2FN	1FN + depender de la clave completa	el precio depende solo de <code>producto_id</code> , no del pedido
3FN	2FN + sin depender de una columna que no es la clave	el nombre del cliente se llega pasando por <code>cliente_id</code>
FNBC	3FN + todo determinante es clave candidata	hay dos claves candidatas que se pisan

¿Esto se usa en la práctica?

Casi nadie dicta “normalizo hasta FNBC”. Lo que sí se hace todo el tiempo: diseñar para que cada dato se escriba **una sola vez**, que es exactamente a dónde llega la 3FN.

¿Y desnormalizar? Cuando sí conviene repetir un dato

Normalizar evita la redundancia, pero tiene un costo: para armar el pedido completo hay que **unir** cuatro tablas.

Normalizado (lo que hicimos hoy)

Un solo lugar para corregir cada dato. Ideal para **registrar** ventas todos los días.

Desnormalizado (a propósito)

Copiar `producto_nombre` dentro de `detalle_pedido` para no unir tablas en un reporte que se lee **miles de veces**.

Cómo se ve en la realidad

Es una decisión, no un error: se normaliza por defecto y se desnormaliza **a propósito**, casi siempre en reportes o paneles donde la velocidad de lectura importa más que ahorrar espacio.

- ▶ Una tabla está en 1FN cuando _____
- ▶ Una tabla está en 2FN cuando, además de 1FN, _____
- ▶ Una tabla está en 3FN cuando, además de 2FN, _____
- ▶ La forma normal que alcanza casi siempre en la práctica es _____

Coronel, C., Morris, S., & Rob, P. (2011). *Bases de datos: diseño, implementación y administración* (9.^a ed.). Cengage Learning.

Elmasri, R., & Navathe, S. (2007). *Fundamentos de sistemas de bases de datos* (5.^a ed.). Addison Wesley.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database system concepts* (7th ed.). McGraw-Hill.



gracias



Universidad
Tecnológica
del Perú